In [2]:
```
import numpy as np
import pandas as pd
import time
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score
```

In [3]:
```
mnist=fetch_openml('mnist_784',version=1,as_frame=False)
X=mnist.data.astype("float32")
y=mnist.target.astype(int)
```

In [4]:
```
X=X/255.0
X=X[:20000]
y=y[:20000]
print("Dataset shape:",X.shape)
```

```
Dataset shape: (20000, 784)
```

In [5]:
```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
print("Training set size:",X_train.shape[0])
print("Testing set size:",X_test.shape[0])
```

```
Training set size: 16000
Testing set size: 4000
```

In [6]:
```
model_sig=MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='logistic',
        solver='adam',
        max_iter=1000,
        random_state=42
    )
start_time_sig=time.time()
model_sig.fit(X_train,y_train)
training_time_sig=time.time()-start_time_sig
sig_pred=model_sig.predict(X_test[:2000])
print(f"accuracy:",accuracy_score(y_test[:2000],sig_pred))
```

```
accuracy: 0.964
```

In [7]:
```
model_relu=MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='relu',
        solver='adam',
        max_iter=1000,
        random_state=42
    )
start_time_relu=time.time()
model_relu.fit(X_train,y_train)
training_time_relu=time.time()-start_time_relu
relu_pred=model_relu.predict(X_test[:2000])
print(f"accuracy:",accuracy_score(y_test[:2000],relu_pred))
```

```
accuracy: 0.967
```

In [8]:
```
model_tanh=MLPClassifier(
        hidden_layer_sizes=(128,64),
        activation='tanh',
        solver='adam',
        max_iter=1000,
        random_state=42
    )
start_time_tanh=time.time()
model_tanh.fit(X_train,y_train)
training_time_tanh=time.time()-start_time_tanh
tanh_pred=model_tanh.predict(X_test[:2000])
print(f"accuracy:",accuracy_score(y_test[:2000],tanh_pred))
```

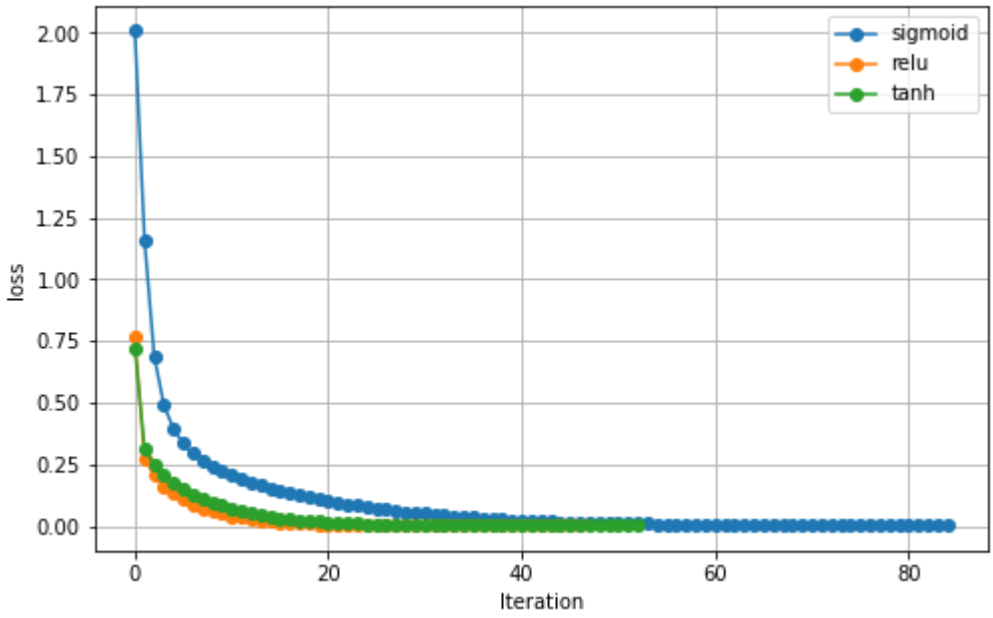
```
accuracy: 0.9645
```

In [9]:
```
print("Training Time:")
print("sigmoid:",training_time_sig)
print("relu:",training_time_relu)
print("tanh:",training_time_tanh)
```

```
Training Time:
sigmoid: 15.787498474121094
relu: 10.836367130279541
tanh: 12.545509099960327
```

In [10]:
```
loss_curves={}
loss_curves["logistic"]=model_sig.loss_curve_
loss_curves["relu"]=model_relu.loss_curve_
loss_curves["tanh"]=model_tanh.loss_curve_
```

In [11]:
```
plt.figure(figsize=(8,5))
plt.plot(loss_curves["logistic"],marker="o",label="sigmoid")
plt.plot(loss_curves["relu"],marker="o",label="relu")
plt.plot(loss_curves["tanh"],marker="o",label="tanh")
plt.xlabel("Iteration")
plt.ylabel("loss")
plt.legend()
plt.grid()
plt.show()
```



In [16]:
```
result_df=pd.DataFrame({'Activation function':['sigmoid','relu','tanh'],'Training ti
result_df.head()
```

Out[16]:

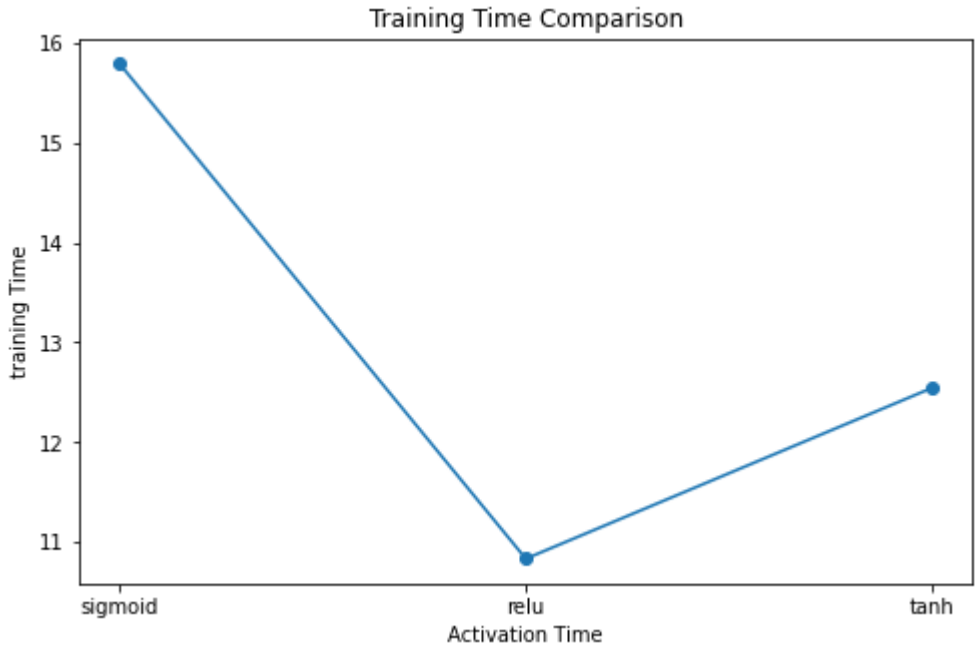
	Activation function	Training time
0	sigmoid	15.787498
1	relu	10.836367
2	tanh	12.545509

In [17]:
```
activation=[]
activation.append('sigmoid')
activation.append('relu')
activation.append('tanh')
training_time=[training_time_sig,training_time_relu,training_time_tanh]
```

In [18]:
```
plt.figure(figsize=(8,5))

plt.plot(
    activation,
    training_time,
    marker="o",
    label="Training time",
)

plt.title("Training Time Comparison")
plt.xlabel("Activation Time")
plt.ylabel("training Time")
plt.show()
```



In [ ]:
```
```